

Visualizing Various Non linear Equations Set I

Deepali Pandey

November 17, 2025

Problem 1:

As an example of homoclinic bifurcation we have studied the dynamical system:

$$\begin{aligned}\dot{x} &= y \\ \dot{y} &= -\mu - x + x^2 - xy\end{aligned}$$

Vary the parameter and draw the phase portrait with vector fields and trajectories to show: no fixed points, stable fixed point, center + saddle, unstable, spiral + saddle

Solution:

Fixed point occurs at $\dot{x} = 0$ and $\dot{y} = 0$. Hence setting up the equation to zero.

$$\begin{aligned}\dot{x} &= y = 0 \\ \dot{y} &= -\mu - x + x^2 - xy = 0\end{aligned}$$

We get $y = 0$ and substituting $y = 0$ in equation of $\dot{y}$ we get $x^2 - x - \mu = 0$. Solving the equation for x :

$$\begin{aligned}(x_1, y) &= \left(\frac{1 + \sqrt{1 + 4\mu}}{2}, 0 \right) \\ (x_2, y) &= \left(\frac{1 - \sqrt{1 + 4\mu}}{2}, 0 \right)\end{aligned}$$

For Stability analysis we find Jacobian:

$$\begin{vmatrix} \frac{\partial \dot{x}}{\partial x} & \frac{\partial \dot{x}}{\partial y} \\ \frac{\partial \dot{y}}{\partial x} & \frac{\partial \dot{y}}{\partial y} \end{vmatrix} = \begin{vmatrix} 0 & 1 \\ -1 + 2x - y & -x \end{vmatrix}$$

With Trace (τ) = $-x$ and Determinant (Δ) = $1 - 2x + y$ and $\tau^2 - 4\Delta$ gives the classification of fixed points.

Hence for various values of μ the stability is as follows:

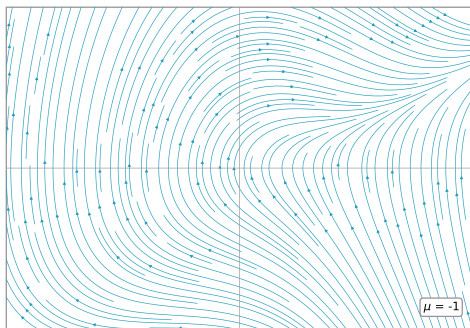
- $\mu < 0$ gives no fixed points.
- $\mu = 0$ give Center with a saddle
- $\mu = 0.5$ gives Stable fixed point
- $\mu = 1$ gives Unstable spiral with saddle

The numerical visualization for various fixed point can be represented by the following Python code

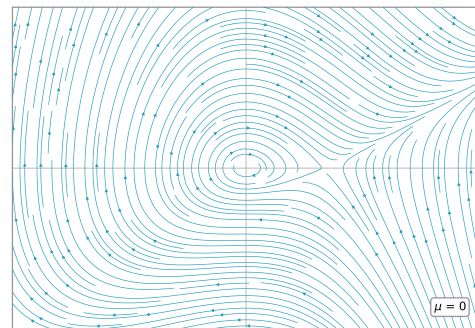
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-3, 3, 100)
5 y = np.linspace(-3, 3, 100)
6
7 X,Y = np.meshgrid(x, y)
8
9 mu = 0 #Adjust as per fixed point
10 Xdot = Y
11 Ydot = -mu - X + X**2 - X*Y
12
13 fig, ax = plt.subplots(1, 1, figsize=(13, 9))
14
15 ax.streamplot(X,Y, Xdot, Ydot, color = '#219ebc',
16               linewidth = 1, broken_streamlines=True, density = 2)
17
18 plt.show()

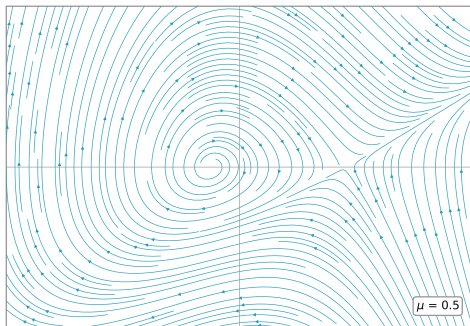
```



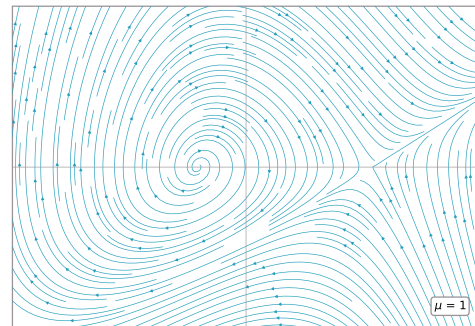
(a) $\mu = -1$ ($\mu < 0$); fixed points



(b) $\mu = 0$; Center with saddle



(c) $\mu = 0.5$ Stable fixed point



(d) $\mu = 1$; Unstable spiral with saddle

Figure 1: Variation in μ values shown as phase portrait: no fixed points, center + saddle, stable fixed point, unstable spiral + saddle

Problem 2:

Consider a dynamical system:

$$\begin{aligned}\dot{v} &= v - \frac{v^3}{3} - w + RI_{ext} \\ \tau \dot{w} &= v + a - bw\end{aligned}$$

where and let $a = 0.7$, $b = 0.8$, $\tau = 12.5$, $R = 0.1$, $I_{ext} \in [-30 : +30]$

Vary I_{ext} to show that a limit cycle emerges for an intermediate range of values. Plot nullclines, mark the fixed points, underlying vector field, and the limit cycle. In the same plot show a few trajectories (starting with different initial conditions) converging to the limit cycle. Approximately estimate the time periods (t) of the limit cycles and plot $TvsI_{ext}$.

Solution:

The dynamical system is given as

$$\begin{aligned}\dot{v} &= v - \frac{v^3}{3} - w + RI_{ext} \\ \tau \dot{w} &= v + a - bw\end{aligned}$$

where and let $a = 0.7$, $b = 0.8$, $\tau = 12.5$, $R = 0.1$, $I_{ext} \in [-30 : +30]$

The nullclines are curves where either $\dot{v} = 0$ or $\dot{w} = 0$,

Nullcline for $\dot{v} = 0$:

$$\begin{aligned}\dot{v} &= v - \frac{v^3}{3} - w + RI_{ext} = 0 \\ w &= v - \frac{v^3}{3} + RI_{ext}\end{aligned}$$

Nullcline for $\dot{w} = 0$:

$$\begin{aligned}\dot{w} &= \frac{v + a - bw}{\tau} = 0 \\ w &= \frac{v + a}{b}\end{aligned}$$

Fixed points are where the nullclines intersect. This occurs when:

$$v - \frac{v^3}{3} - w + RI_{ext} = \frac{v + a}{b}$$

We can solve this numerically to find the values of v and w that satisfy both equations. In case of Nullcline for $\dot{v} = 0$ the equation changes as it has dependence on I_{ext} ; whereas the equation for $\dot{w} = 0$ is independent of I_{ext} . Hence we need to vary the I_{ext} to see dynamical evolution of fixed point. The following python code plots both Nullcline, fixed points, vector field for various values of I_{ext} . For finding the limit cycle we numerically integrate the equation.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4 from scipy.integrate import solve_ivp
5
6 a = 0.7
7 b = 0.8
8 tau = 12.5
9 R = 0.1
10
11 def system(t, y, I_ext):
12     v, w = y
13     dv_dt = v - (v**3) / 3 - w + R * I_ext
14     dw_dt = (v + a - b * w) / tau
15     return [dv_dt, dw_dt]
16
17 t_span = [0, 100]
18 t_eval = np.linspace(t_span[0], t_span[1], 1000)
19
20 initial_conditions = [[-1.0, 1.0], [0.5, 0.0], [-0.5, -1.0]]
21 I_ext_values = np.arange(-30, 32, 2)
22 colors = plt.cm.winter(np.linspace(0, 1, len(I_ext_values)))
23 v_grid = np.linspace(-3, 3, 20)
24 w_grid = np.linspace(-3, 3, 20)
25 V, W = np.meshgrid(v_grid, w_grid)
26
27 fig, axes = plt.subplots(8, 4, figsize=(70, 80), constrained_layout=True)
28 axes = axes.flatten()
29
30 for idx, I_ext in enumerate(I_ext_values):
31     if idx >= len(axes):
32         break
33     ax = axes[idx]
34     color = colors[idx]
35     dv_dt = V - (V**3) / 3 - W + R * I_ext
36     dw_dt = (V + a - b * W) / tau
37     ax.quiver(V, W, dv_dt, dw_dt, color='#219ebc', alpha=0.6, scale=30, angles='xy',
38              scale_units='xy', headwidth = 3)
39     ax.plot(v_vals, w_nullcline(v_vals), color='#CD5C08', linewidth=2)
40
41     v_fp = fsolve(fixed_points, [-2, 0, 2], args=(I_ext))
42     w_fp = w_nullcline(v_fp)
43
44     v_vals = np.linspace(-3, 2, 400)
45     w_v_nullcline = v_nullcline(v_vals, I_ext)
46     ax.plot(v_vals, w_v_nullcline, color='#347928', linewidth=2)
47     ax.plot(v_fp, w_fp, marker = '+', ms = 20, mec = '#347928')
48     for v, w in zip(v_fp, w_fp):
49         ax.annotate(f"({v:.2f}, {w:.2f})", xy=(v, w), xytext=(10, -10),
50                   textcoords='offset points', fontsize=20, color='black', bbox=dict(boxstyle=
51 "round", fc="w", color = '#9A9498', lw = 0.5))
52
53     for ic in initial_conditions:
54         sol = solve_ivp(system, t_span, ic, args=(I_ext,), t_eval=t_eval)
55         ax.plot(sol.y[0], sol.y[1], color=color, linewidth=4)
56
57     ax.annotate(f"$I_{\{ext\}}=\{I_ext\}$", xy=(0.97, 0.05), xycoords='axes fraction', ha='right',
58               va='bottom', fontsize=30, bbox=dict(boxstyle="round", fc="w", color = '#9A9498', lw = 2))
59
60 fig.text(0.5, -0.02, 'v', ha='center', fontsize=50)
61 fig.text(-0.04, 0.5, 'w', va='center', rotation='vertical', fontsize=50)
62
63 if len(I_ext_values) < len(axes):
64     for i in range(len(I_ext_values), len(axes)):
65         fig.delaxes(axes[i])
66
67 plt.show()

```

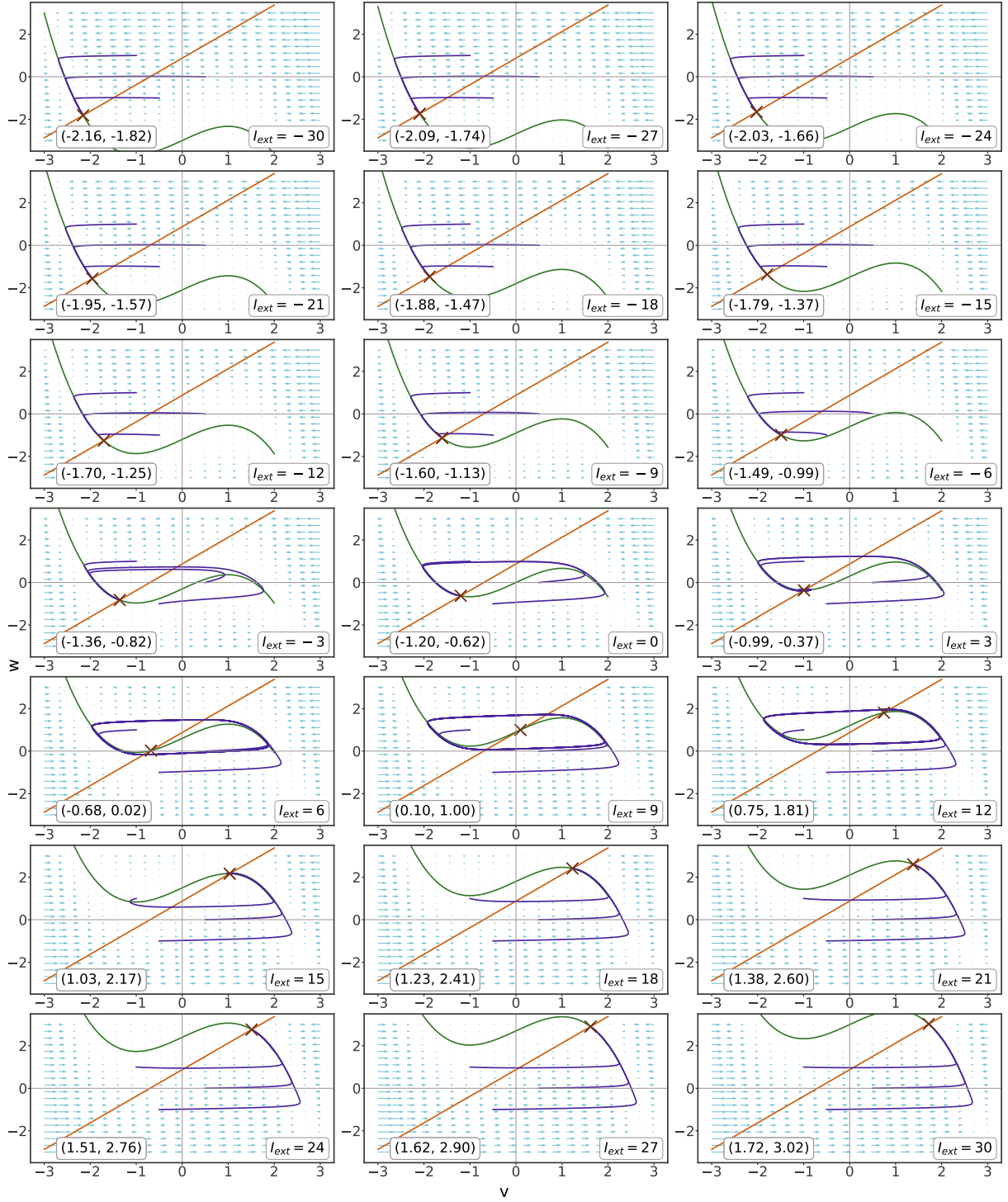



Figure 2: Dynamics of a two-dimensional system under varying external current (I_{ext}). The plot illustrates the phase portraits of the dynamical system characterized by the equations of motion v and w for selected values of I_{ext} ranging from -30 to 30 in increments of 3. Each subplot displays, the vector field (Blue arrows) representing the direction and magnitude of the system's dynamics. The annotations indicate the corresponding I_{ext} value for each subplot. The emergence of limit cycles (plotted in purple) can be observed in specific ranges of I_{ext} , highlighting the sensitivity of the system's behavior to external perturbations. The Orange and Green line represent the Nullcline for $\dot{w}$ and $\dot{v}$ respectively and the 'X' denotes their intesection, which is a fixed point. We can see from the plot that the limit cycle emerges from $-6 < I_{ext} < 15$

To approximately estimate the time periods (t) of the limit cycles and plot $T_{vsI_{ext}}$ we use the following python code. For a range of values, simulate the system using an appropriate initial condition., then we identify the peaks of the v variable, which indicates the oscillatory behavior of the system. We calculate the period T based on the time difference between consecutive peaks.

```

1 import numpy as np
2 import scienceplots
3 import matplotlib.pyplot as plt
4 from scipy.optimize import fsolve
5 from scipy.integrate import solve_ivp
6 from scipy.signal import find_peaks
7
8 a = 0.7
9 b = 0.8
10 tau = 12.5
11 R = 0.1
12
13 def system(t, y, I_ext):
14     v, w = y
15     dv_dt = v - (v**3) / 3 - w + R * I_ext
16     dw_dt = (v + a - b * w) / tau
17     return [dv_dt, dw_dt]
18
19 t_span = [0, 200]
20 t_eval = np.linspace(t_span[0], t_span[1], 1000)
21 I_ext_values = np.arange(-30, 31, 1)
22
23 periods = []
24
25 for I_ext in I_ext_values:
26     initial_condition = [0.5, 0.0]
27     sol = solve_ivp(system, t_span, initial_condition, args=(I_ext,), t_eval=t_eval)
28     peaks, _ = find_peaks(sol.y[0])
29
30     if len(peaks) >= 2:
31         period = np.mean(np.diff(sol.t[peaks]))
32         periods.append(period)
33     else:
34         periods.append(np.nan)
35
36
37
38 fig, ax = plt.subplots(1, 1, figsize=(13, 9))
39
40 ax.plot(I_ext_values, periods, marker='o', linestyle='-', color = '#1d4e89')
41 ax.set_ylabel('Time Period  $T$ ', fontweight='normal', fontsize=20)
42 ax.set_xlabel(' $I_{ext}$ ', fontweight='normal', fontsize=20)
43 plt.show()

```

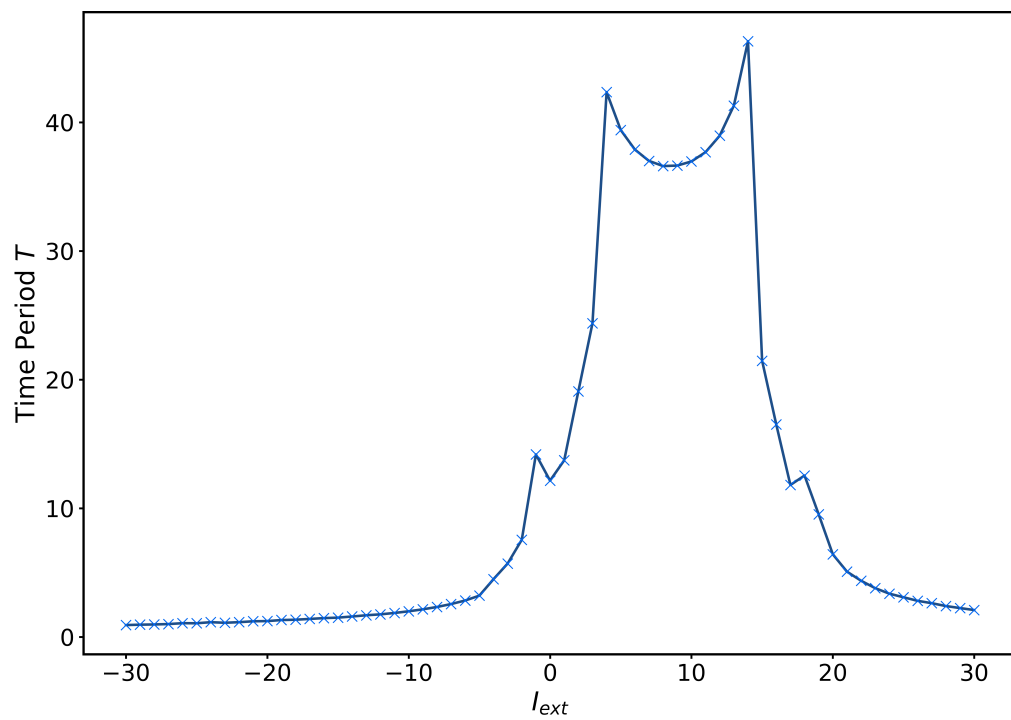


Figure 3: Here is the plot of the time period T of the limit cycles as a function of I_{ext} . This range reflects the values where closed orbits (limit cycles) are expected based on observation in Fig 2. The peaks in the graph indicate the estimated period of the limit cycles at each value of

Problem 3:

Numerically simulate the Lorenz system of equations:

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= rx - y - xz \\ \dot{z} &= xy - bz\end{aligned}$$

for $r = 28, \sigma = 10, b = \frac{8}{3}$

Solution

(a) Show the 3D attractor in (x, y, z) plot:

Python code:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4 from scipy.integrate import solve_ivp
5 from mpl_toolkits.mplot3d import Axes3D
6 import matplotlib.cm as cm
7
8
9 def lorenz(t, state, sigma, r, beta):
10     x, y, z = state
11     dxdt = sigma * (y - x)
12     dydt = x * (r - z) - y
13     dzdt = x * y - beta * z
14     return [dxdt, dydt, dzdt]
15
16 sigma = 10.0
17 r = 28.0
18 beta = 8.0 / 3.0
19
20 initial_state = [1.0, 1.0, 1.0]
21
22 t_span = (0, 100)
23 t_eval = np.linspace(t_span[0], t_span[1], 100000)
24
25 sol = solve_ivp(lorenz, t_span, initial_state, args=(sigma, r, beta), t_eval=t_eval)
26
27 x = sol.y[0]
28 y = sol.y[1]
29 z = sol.y[2]
30 t = sol.t
31
32 fig = plt.figure(figsize=(13, 9))
33 ax = fig.add_subplot(111, projection='3d')
34
35
36 norm = plt.Normalize(t.min(), t.max())
37 colors = cm.summer(norm(t))
38
39
40 for i in range(len(t) - 1):
41     ax.plot(x[i:i+2], y[i:i+2], z[i:i+2], color=colors[i], lw=0.5)
42
43 ax.set_xlabel('X', fontweight='normal', fontsize=20)
44 ax.set_ylabel('Y', fontweight='normal', fontsize=20)
45 ax.set_zlabel('Z', fontweight='normal', fontsize=20)
46
47 ax.text2D(0.05, 0.95, "Initial State = (1.0, 1.0, 1.0)", transform=ax.transAxes, bbox=dict(
48     boxstyle="round", fc="w", color = '#353935', lw = 1))
49
50 ax.spines['bottom'].set_linewidth(1.8)
51 ax.spines['left'].set_linewidth(1.8)
52 ax.spines['top'].set_linewidth(1.8)
53 ax.spines['right'].set_linewidth(1.8)
54 ax.spines['bottom'].set_color('#353935')
```

```

54 ax.spines['left'].set_color('#353935')
55 ax.spines['top'].set_color('#353935')
56 ax.spines['right'].set_color('#353935')
57 ax.set_xticklabels([])
58 ax.set_yticklabels([])
59 ax.set_zticklabels([])
60 ax.grid(False)
61 for label in ax.get_xticklabels() + ax.get_yticklabels():
62     label.set_fontweight('normal')
63     label.set_color('#353935')
64
65
66 ax.xaxis.pane.fill = False
67 ax.yaxis.pane.fill = False
68 ax.zaxis.pane.fill = False
69
70
71 sm = plt.cm.ScalarMappable(cmap=cm.summer, norm=norm)
72 sm.set_array([])
73 cbar = fig.colorbar(sm, ax=ax, shrink=0.5, aspect=10)
74 cbar.set_label('Time', fontweight='normal', fontsize=20)
75 cbar.set_ticks([])
76
77 plt.show()

```

Initial State = (1.0, 1.0, 1.0)

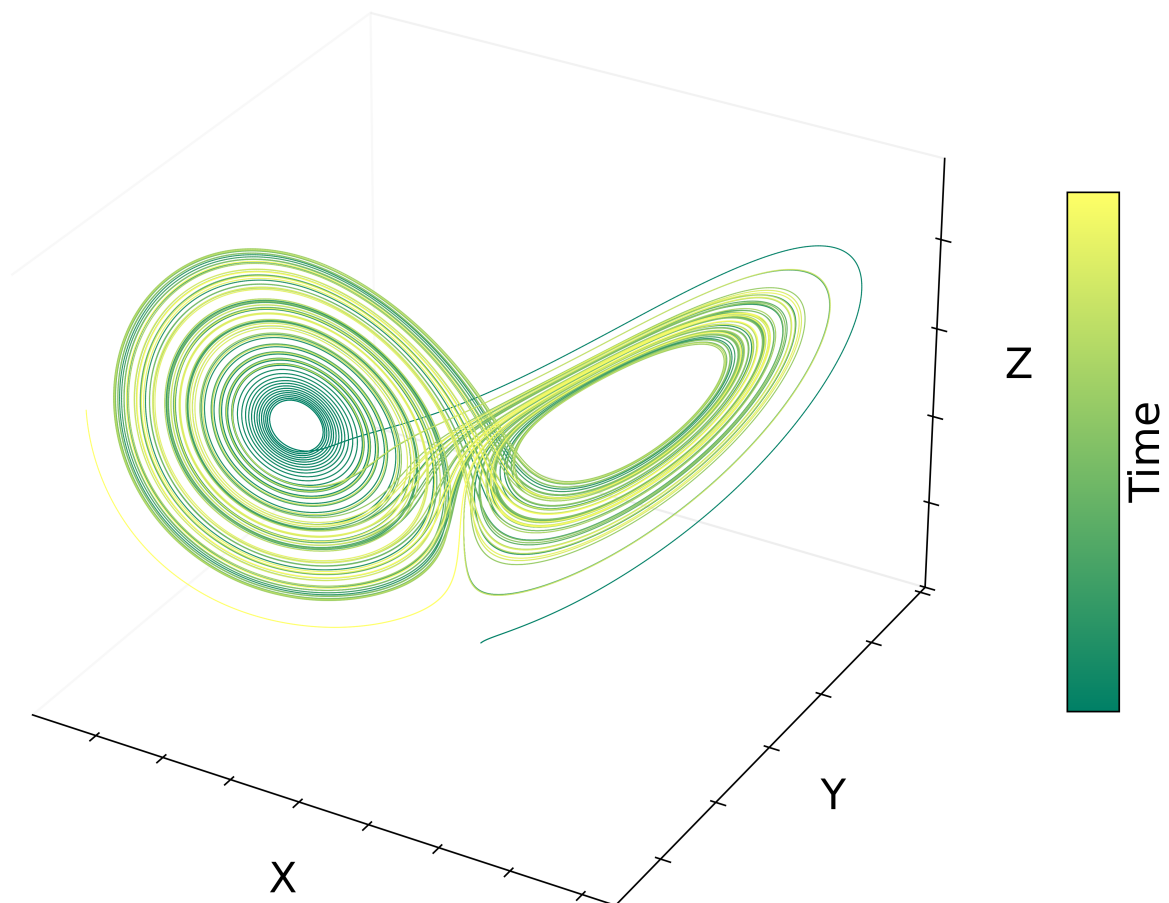


Figure 4: 3D attractor in xyz plot

(b) show sensitive dependence on initial conditions by choosing a pair of initial conditions differing by a small amount we choose initial condition as (1.0, 1.0, 1.0) and (1.0001, 1.0, 1.0), plot the attractor map and compute the difference $x_1(t) - x_2(t)$

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4 from scipy.integrate import solve_ivp
5 from mpl_toolkits.mplot3d import Axes3D
6 import matplotlib.cm as cm
7
8 def lorenz(t, state, sigma, rho, beta):
9     x, y, z = state
10    dxdt = sigma * (y - x)
11    dydt = x * (rho - z) - y
12    dzdt = x * y - beta * z
13    return [dxdt, dydt, dzdt]
14
15 sigma = 10.0
16 rho = 28.0
17 beta = 8.0 / 3.0
18
19 initial_state_1 = [1.0, 1.0, 1.0]
20 initial_state_2 = [1.0001, 1.0, 1.0]
21
22 t_span = (0, 50)
23 t_eval = np.linspace(t_span[0], t_span[1], 10000)
24
25 sol1 = solve_ivp(lorenz, t_span, initial_state_1, args=(sigma, rho, beta), t_eval=t_eval)
26 sol2 = solve_ivp(lorenz, t_span, initial_state_2, args=(sigma, rho, beta), t_eval=t_eval)
27
28 x1, y1, z1 = sol1.y[0], sol1.y[1], sol1.y[2]
29 x2, y2, z2 = sol2.y[0], sol2.y[1], sol2.y[2]
30
31 fig = plt.figure(figsize=(13, 13))
32 ax = fig.add_subplot(111, projection='3d')
33
34 ax.plot(x1, y1, z1, color = '#219ebc', linewidth = 1.8, label='Initial condition (1.0, 1.0, 1.0)')
35 ax.plot(x2, y2, z2, color = '#ef233c', linewidth = 1.8, label='Initial condition (1.0001, 1.0, 1.0)')
36
37 ax.set_xlabel('X', fontweight='normal', fontsize=20, labelpad=15)
38 ax.set_ylabel('Y', fontweight='normal', fontsize=20, labelpad=15)
39 ax.set_zlabel('Z', fontweight='normal', fontsize=20, labelpad=15)
40
41 plt.show()
42
43 %-----%
44
45 diff = np.sqrt((x1 - x2)**2 + (y1 - y2)**2 + (z1 - z2)**2)
46 fig, ax = plt.subplots(1, 1, figsize=(13, 8))
47 ax.plot(t_eval, diff, lw=2, color = '#011288')
48 ax.set_xlabel('Time', fontweight='normal', fontsize=30, color = '#353935')
49 ax.set_ylabel("$x_1(t) - x_2(t)$", fontweight='normal', fontsize=30, color = '#353935')
50
51 ax.set_yscale('log')
52 plt.show()

```

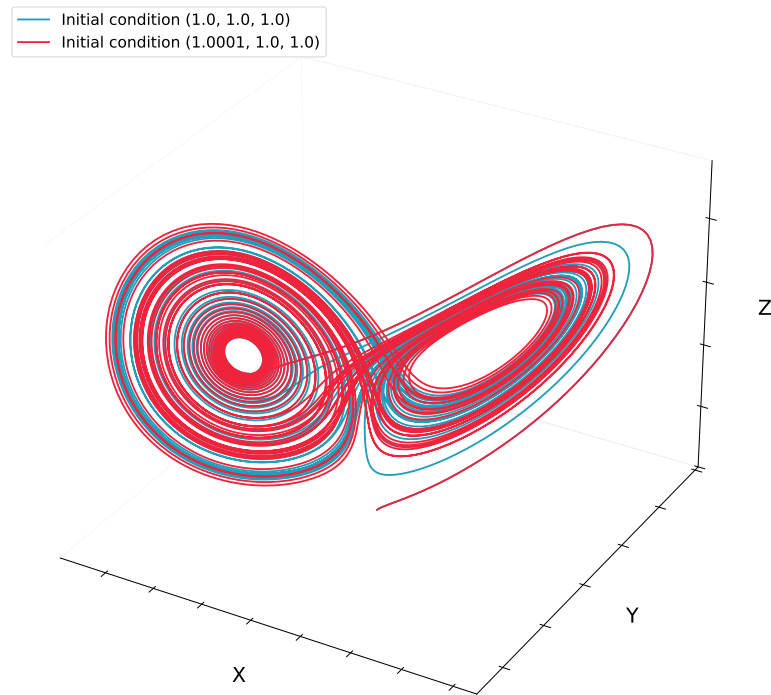


Figure 5: sensitive dependence on initial conditions by choosing initial condition as (1.0, 1.0, 1.0) and (1.0001, 1.0, 1.0)

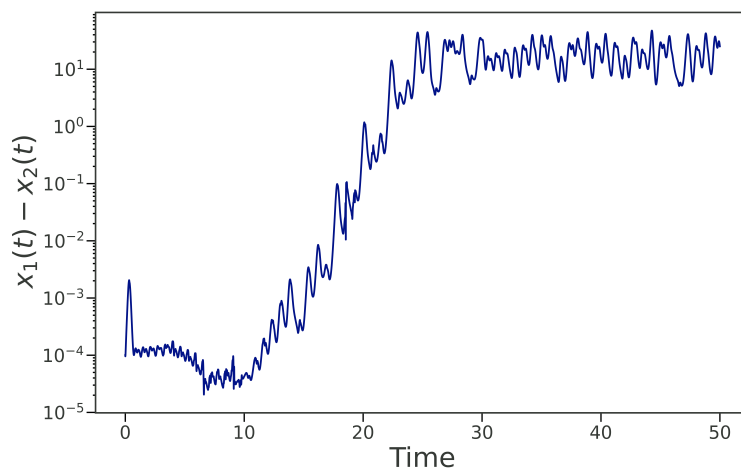


Figure 6: The difference graph depicts the divergence of two Lorenz system trajectories that started from almost similar initial conditions. Despite starting close together, the paths diverge exponentially with time, showing the sensitive dependency on initial conditions that characterizes chaotic systems. The logarithmic scale on the y-axis emphasizes this difference, illustrating how minor changes in initial conditions can result in significantly different outcomes.

(c) To find the local maximum of z , plot the first return map ($\sim$ Poincare map) and numerically determine $\bar{z}$ such that $\bar{Z} = P(\bar{Z})$ we do the following:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4 from scipy.integrate import solve_ivp
5 from scipy.signal import argrelextrema
6
7 sigma = 10
8 b = 8/3
9 r = 28
10
11 def lorenz_system(dt, steps, initial_conditions):
12     x, y, z = initial_conditions
13     trajectory = []
14     for _ in range(steps):
15         dx = sigma * (y - x) * dt
16         dy = (x * (r - z) - y) * dt
17         dz = (x * y - b * z) * dt
18         x += dx
19         y += dy
20         z += dz
21         trajectory.append((x, y, z))
22     return np.array(trajectory)
23
24 dt = 0.01
25 steps = 100000
26 initial_conditions = (1.0, 1.0, 1.0)
27 trajectory = lorenz_system(dt, steps, initial_conditions)
28
29 z_values = trajectory[:, 2]
30
31 local_max_indices = argrelextrema(z_values, np.greater)[0]
32 local_maxima = z_values[local_max_indices]
33
34 z_return_map = local_maxima[:-1]
35 z_next = local_maxima[1:]
36
37 fig, ax = plt.subplots(1, 1, figsize=(13, 12))
38 ax.scatter(z_return_map, z_next, s=20, color = '#011288')
39 ax.axline((0, 0), slope=1, color='#ef233c', linestyle='--', label='y = x', lw = 2)
40 ax.set_xlabel(r'$z_n$', fontweight='normal', fontsize=30, color = '#353935')
41 ax.set_ylabel(r'$z_{n+1}$', fontweight='normal', fontsize=30, color = '#353935')
42 plt.show()

```

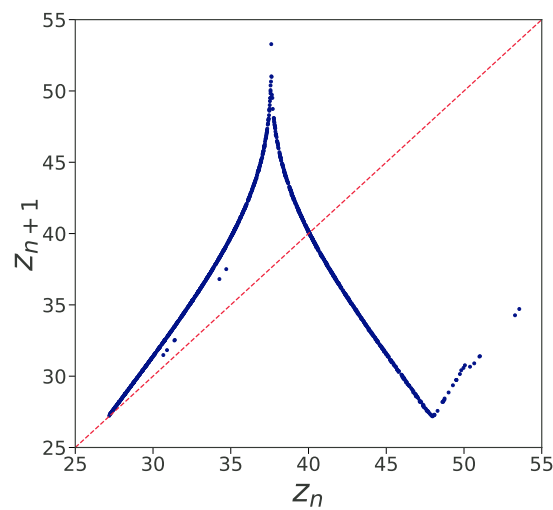


Figure 7: First Return Map for Lorentz Equation

To demonstrate the existence of an unstable periodic orbit at $\bar{z}$ in the Lorenz system, we first identify the local maxima of the z -coordinate and construct a first return map, plotting z_n against z_{n+1} . A fixed point $\bar{z}$ of the return map satisfies $P(\bar{z}) = \bar{z}$, representing a periodic orbit in the continuous system. We numerically determine $\bar{z}$ by solving $P(z) = z$ and estimate the slope $P'(\bar{z})$ at this point using finite differences. The magnitude of the slope $|P'(\bar{z})|$ indicates the stability of the orbit. Since $|P'(\bar{z})| > 1$, small deviations from $\bar{z}$ grow, confirming that the periodic orbit is unstable, which is a characteristic feature of chaotic systems like the Lorenz attractor.

```

1 import numpy as np
2 from scipy.signal import argrelextrema
3 from scipy.interpolate import interp1d
4 from scipy.optimize import fsolve
5
6 sigma = 10
7 b = 8/3
8 r = 28
9
10 def lorenz_system(dt, steps, initial_conditions):
11     x, y, z = initial_conditions
12     trajectory = []
13     for _ in range(steps):
14         dx = sigma * (y - x) * dt
15         dy = (x * (r - z) - y) * dt
16         dz = (x * y - b * z) * dt
17         x += dx
18         y += dy
19         z += dz
20         trajectory.append((x, y, z))
21     return np.array(trajectory)
22
23 dt = 0.01
24 steps = 10000
25 initial_conditions = (1.0, 1.0, 1.0)
26 trajectory = lorenz_system(dt, steps, initial_conditions)
27
28 z_values = trajectory[:, 2]
29
30 local_max_indices = argrelextrema(z_values, np.greater)[0]
31 local_maxima = z_values[local_max_indices]
32
33 z_return_map = local_maxima[:-1]
34 z_next = local_maxima[1:]
35
36 return_map_func = interp1d(z_return_map, z_next, kind='cubic', fill_value="extrapolate")
37
38 def fixed_point_function(z):
39     return return_map_func(z) - z
40
41 initial_guess = np.mean(local_maxima)
42 fixed_point = fsolve(fixed_point_function, initial_guess)[0]
43
44 print(f"Numerically determined fixed point z: {fixed_point:.4f}")
45
46 def derivative_at_fixed_point(z):
47     h = 1e-5
48     return (return_map_func(z + h) - return_map_func(z - h)) / (2 * h)
49
50 slope_at_fixed_point = derivative_at_fixed_point(fixed_point)
51 print(f"Slope at fixed point (P'(z)): {slope_at_fixed_point:.4f}")
52
53 if abs(slope_at_fixed_point) > 1:
54     print("An unstable periodic orbit exists at z, because |P'(z)| > 1.")
55 else:
56     print("The periodic orbit is stable, because |P'(z)| <= 1.")

```